

Operating Systems — Midterm Practice Exam

Name: _____ Student ID: _____ Date: _____

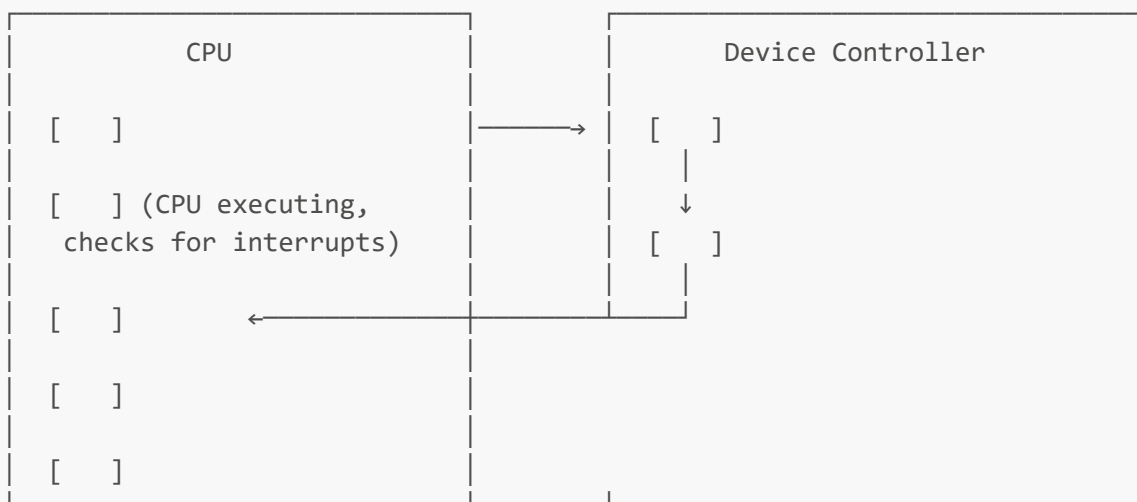
Total: 100 points. Time allowed: 90 minutes. For code fill-in questions, write your answers in the blank table beneath each program. Do **not** modify the given code skeleton.

Q1 Interrupt, I/O Operation (25 pts)

(a) (5 pts) Show the advantage of interrupt-based operations over busy-waiting.

Answer:

(b) (10 pts) The interrupt-driven I/O cycle consists of (i) Input ready, output complete, or error generates interrupt signal, (ii) Interrupt handler processes data, return from interrupt, (iii) CPU executing checks for interrupts between instructions, (iv) device driver initiates I/O, (v) Controller initiates I/O, (vi) CPU resumes processing of interrupted process (task), and (vii) CPU receiving interrupt, transfer control to interrupt handler. **Organize** the steps with the following flowchart.



(c) (5 pts) What role do device controllers and device drivers play in a computer system?

Device controller:

Device driver:

(d) (5 pts) Explain the benefit of using interrupt vector.

Answer:

Q2 System Calls, Operating System Design (20 pts)

(a) (10 pts) Describe how a system call is invoked using an API function? You **need to mention software interrupt, dual-mode, system call number (index), and table of code pointer.**

Answer:

(b) (5 pts) Explain why programmers prefer programming according to API functions rather than invoking system calls directly.

Answer:

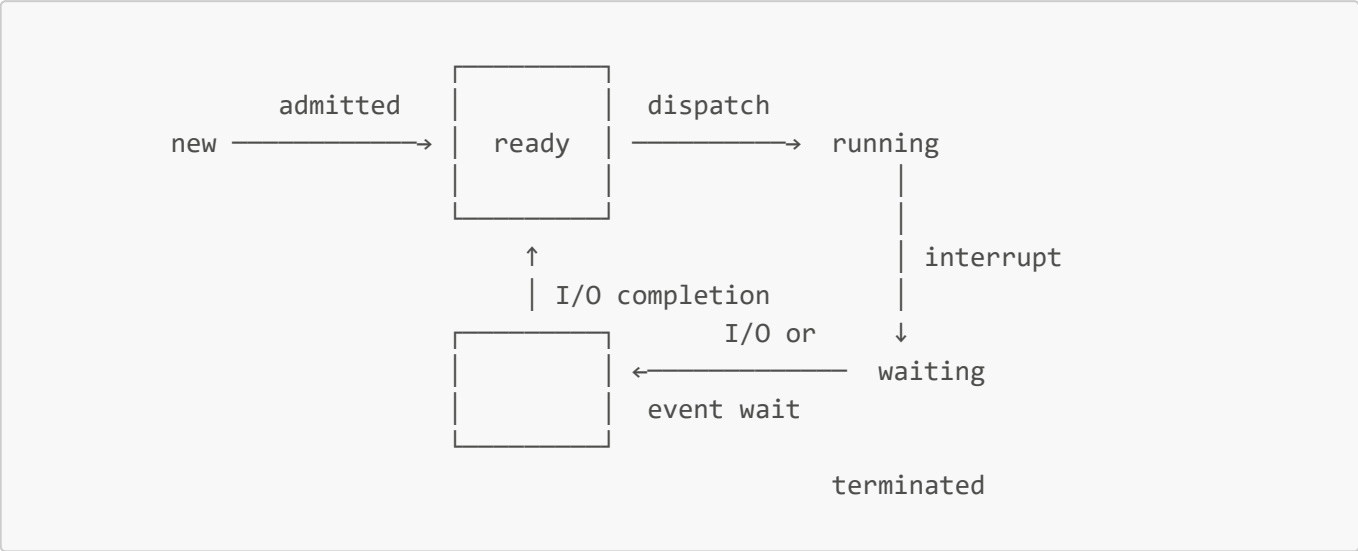
(c) (5 pts) Describe the pros and cons of layered operating system structure.

Pros:

Cons:

Q3 Process State (20 pts)

(a) (10 pts) During its life cycle, a process can (i) be interrupted, (ii) be dispatched by a short-term scheduler, (iii) perform I/O, (iv) have I/O completed, (v) exit. Organize these operations with the following diagram.



Fill in each arrow label with the correct operation from (i)–(v):

admitted → ready : ()

ready → running : ()

running → ready : ()

running → waiting : ()

waiting → ready : ()

running → terminated : ()

(b) Show the state of a process in the following situations: **(10 pts)**

#	Situation	State
i	after it is created (forked) by a (parent) process	
ii	before it is selected by a short-term scheduler	
iii	after it is selected by a short-term scheduler	
iv	before it invokes a blocking system call	
v	after it invokes a blocking system call	
vi	after the blocking system call is complete	
vii	after it is interrupted by a hardware interrupt	
viii	after its time slice expired	

Q4 Multithreaded Programming (20 pts)

(a) (5 pts) Explain why allocating memory for process creation is **costly** as compared with thread creation.

Answer :

(b) (5 pts) Compare and contrast the many-to-one and one-to-one thread models.

Many-to-one:

One-to-one:

(c) (5 pts) What is the problem of many-to-many thread model that scheduler activation tries to resolve?

Answer :

(d) (5 pts) Show how scheduler activations work when a user thread is making a blocking system call. You **HAVE TO** mention **upcall handler**, **scheduler**, and **thread library**.

Answer :

Q5 Code Fill-in (15 pts)

Part A — Shared Memory (8 pts)

Please complete the following program that calculates equation $2x + 3y - 5$. The values of variables x and y are provided by users in the command line, i.e., $x = \text{atoi}(\text{argv}[1])$ and $y = \text{atoi}(\text{argv}[2])$.

Hint: the main program first allocates a shared memory and then creates two child processes — one for the x term calculation and one for the y term calculation. Save the results calculated in the child processes in the shared memory and print the final outcome in the parent process.

```
#include <sys/shm.h>
#include <sys/stat.h>
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/wait.h>

int main(int argc, char *argv[]) {
    int x = atoi(argv[1]);
    int y = atoi(argv[2]);
    int i;
    pid_t pid;
    const int sh_size = 4;

    int segment_id = __ (A) __ (IPC_PRIVATE, sh_size, S_IRUSR | S_IWUSR);

    int *shared_memory = (int *) __ (B) __ (segment_id, NULL, 0);

    *shared_memory = __ (C) __;

    for (i = 2; i > 0; i--) {
        pid = __ (D) __ ();
        if (pid < 0) {
            fprintf(stderr, "Fork Failed");
            exit(-1);
        } else if (pid == 0) {
            if (i == 2) *shared_memory += __ (E) __;
            if (i == 1) *shared_memory += __ (F) __;
            shmdt(shared_memory);
            exit(0);
        } else {
            __ (G) __ (NULL);
        }
    }

    *shared_memory -= 5;
    printf("the result is %d\n", *shared_memory);
    shmdt(shared_memory);
    __ (H) __ (segment_id, IPC_RMID, NULL);
    return 0;
}
```

Blank Answer

Blank	Answer
(A)	
(B)	
(C)	
(D)	
(E)	
(F)	
(G)	
(H)	

Part B — Shell Interface (7 pts)

Complete the following C++ program which serves as a simple shell interface. The shell accepts user commands and then executes each command in a separate process. The shell allows the child process to run in the background (or concurrently) if the command ends with `&`. For simplicity, we assume that users never enter invalid commands and no arguments follow a command.

```
#include <iostream>
#include <unistd.h>
#include <string.h>
#include <sys/wait.h>
using namespace std;

int main(void) {
    char inputBuffer[80];
    bool background;

    while (cin >> inputBuffer) {
        cout << endl << " COMMAND->";
        pid_t pid;

        /* check whether the command ends with '&' */
        if (inputBuffer[strlen(inputBuffer)-1] == '&') {
            background = __ (A) __;
            inputBuffer[strlen(inputBuffer)-1] = '\0';
        } else {
            background = __ (B) __;
        }

        /* fork a child process */
        pid = __ (C) __();

        if (pid == 0) {
            /* child process: execute the command */
            __ (D) __ (inputBuffer);
        }
    }
}
```

```
        } else if (pid > 0) {
            /* parent process */
            if (background == __ (E)__)
                __ (F)__(NULL);
        }

    } /* end of while */
    return 0;
} /* end of main */
```

Blank	Answer
(A)	
(B)	
(C)	
(D)	
(E)	
(F)	

Bonus CPU Scheduling (+10 pts)

Consider the following two processes P_1 and P_2 , each of which consists of two CPU bursts and one I/O burst. Assume that P_1 and P_2 arrive at time 0 and 2 (millisecond) respectively:

Process	First CPU burst	I/O burst	Second CPU burst
P_1	8 ms	4 ms	8 ms
P_2	2 ms	2 ms	16 ms

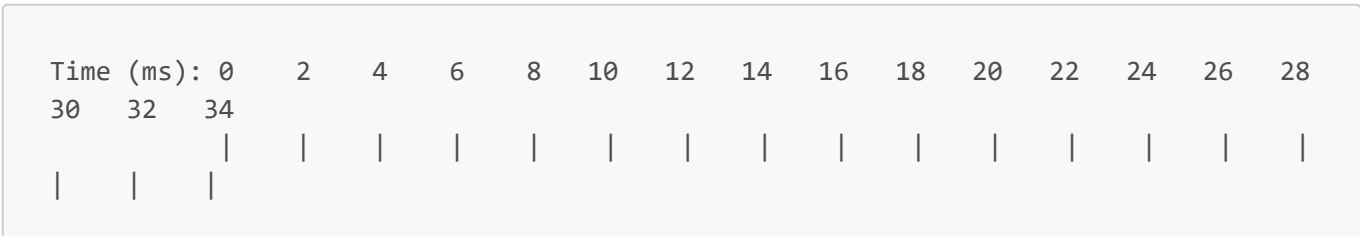
(a) (6 pts) Draw a Gantt chart illustrating the execution of the processes using the following **preemptive multilevel feedback-queue scheduling algorithm**. Note that a high-level process will preempt a low-level process and a process will be scheduled to a lower level if it is preempted.

Round-Robin scheduling with time quantum = 4 ms ← Q2 (highest)

Round-Robin scheduling with time quantum = 6 ms ← Q1

FCFS scheduling ← Q0 (lowest)

A process returning from I/O re-enters **Q2**.



Queue/
Process:

(b) (4 pts) What is the average waiting time for the scheduling algorithm?

Process	Arrival	Finish	Turnaround Time	Waiting Time
P ₁	0			
P ₂	2			

Average Waiting Time = _____ ms

End of exam. Good luck!